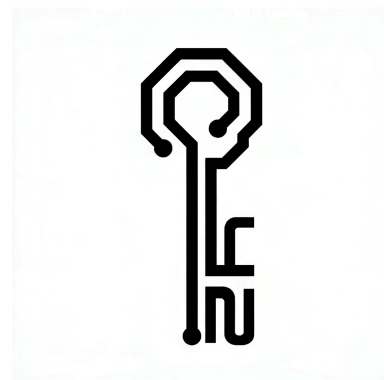


Национальный исследовательский ядерный университет «МИФИ»

Институт интеллектуальных кибернетических систем

Кафедра № 42 «Криптология и кибербезопасность»



ОТЧЕТ

**О выполнении лабораторной работы № 2
«Введение в параллельные вычисления. Технология OpenMP»**

Студент: Доценко В. А.

Группа: Б23-505

Преподаватель: Куприяшин М.А.

Москва — 2025

Оглавление

1. Цель работы	3
2. Рабочая среда	3
3. Анализ последовательного алгоритма	3
3.1. Описание	3
3.2. Оценка	3
3.3. Блок-схема	4
4. Анализ параллельного алгоритма	5
4.1. Описание	5
4.2. Оценка параметров	5
4.3. Директивы OpenMP	5
4.4. Блок-схемы алгоритмов	7
5. Результаты экспериментов	9
6. Заключение	12
7. Приложение	13
7.1. Программные коды	13
7.2. Таблицы	19

1. Цель работы

Приобрести навыки разработки параллельной программы путём обнаружения ресурса параллелизма в имеющейся последовательной реализации.

2. Рабочая среда

Модель процессора: Apple M3.

Количество ядер / потоков: 8 / 8.

Объем оперативной памяти: 24 ГБ.

Разрядность и версия ОС: macOS Sequoia 15.6.

Среда разработки: VS Code, clang 16.0.0.

Версия OpenMP: 4.0 (`_OPENMP = 202011`).

3. Анализ последовательного алгоритма

3.1. Описание

1. Создаётся массив из N элементов.
2. Массив заполняется случайными числами с использованием функции `rand()`.
3. Осуществляется последовательный проход по массиву пока не будет найден искомый элемент.
4. Индекс ячейки массива, содержащей искомый элемент, выводится на экран.

3.2. Оценка

Худшая – полный проход $O(n)$.

Лучшая – первый элемент – искомый $O(1)$.

3.3. Блок-схема

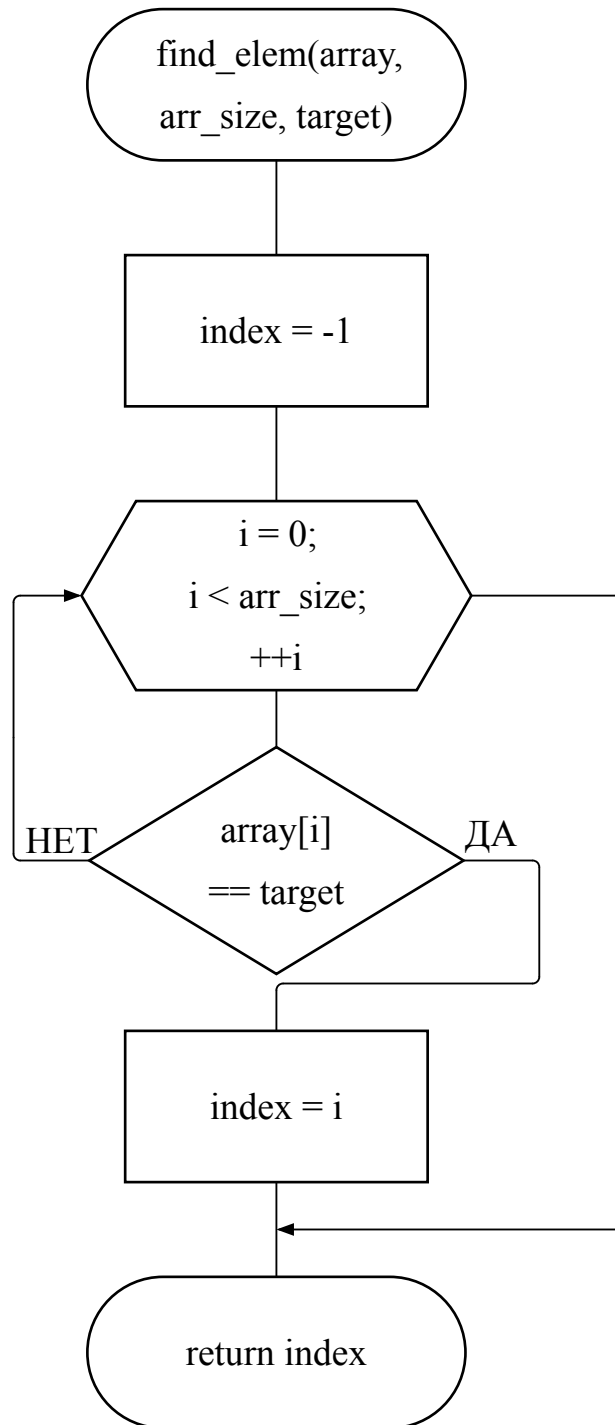


Рисунок 1 – Блок-схема алгоритма, описанного в п. 3.1

4. Анализ параллельного алгоритма

4.1. Описание

Для этого алгоритма я выявил три возможных способа распараллеливания:

1. **Использовать `#pragma omp cancel for` + `#pragma omp critical`.** После того, как индекс найден, происходит отмена оставшихся итераций цикла, потоки мгновенно прекращают свою работу. Теоретически, скорость выполнения этого алгоритма тем выше, чем раньше будет найден подходящий элемент.
2. **Синхронизация через `atomic`.** этот вариант сильно снижает накладные расходы по сравнению с `critical` и `cancel for`, однако потоки будут продолжать свою работу даже после того, как элемент будет найден, будут пропускаться циклы.
3. **Разделение на блоки + редукция минимума индекса.** Полный обход массива с редукцией теоретически должен быть менее эффективным, так как требует больше итераций цикла. Однако, накладные расходы на остановку других потоков могут невелировать полученное преимущество.
4. **Дополнительное задание “`Dirty`”.** При нахождении искомого числа, индекс в цикле `for` выставляется равным максимальному. Таким образом, остальные потоки не начинают новый цикл, т.к. не выполняется условие в `for`

4.2. Оценка параметров

Ускорение: $S_p = \frac{T_1}{T_p}$, где T_1 – время работы в линейном режиме (1 поток), T_p – время работы при p потоках.

Эффективность: $E_p = \frac{S_p}{p}$, где S_p - ускорение, p – количество задействованных потоков.

4.3. Директивы OpenMP

`#pragma omp for`

Директива для распараллеливания цикла `for`. По умолчанию применяется `schedule(static)`, то есть нагрузка распределится равномерно между потоками.

`#pragma omp parallel num_threads(threads) default(none) shared(arr, N, target, index, found)`

Директива, определяющая регион общего доступа к `array`, размера `N`, для

для поиска (совместное чтение) target. При добавлении reduction(min:index) произойдет редукция индекса к минимальному.

#pragma omp critical

Директива, обозначающая критический регион, доступ к которому в каждый момент времени имеет только 1 поток.

#pragma omp cancel for и #pragma omp cancellation point for

Директивы остановки выполнения цикла for в регионе для всех потоков и установки точки для проверки запроса на остановку.

#pragma omp atomic read

Атомарное чтение гарантирует, что поток получит актуальные данные, дешевле, чем чтение через critical.

#pragma omp atomic capture

атомарная комбинация чтения и записи (или другой пары операций) с возможностью «захватить» предыдущее или текущее значение в переменную

4.4. Блок-схемы алгоритмов

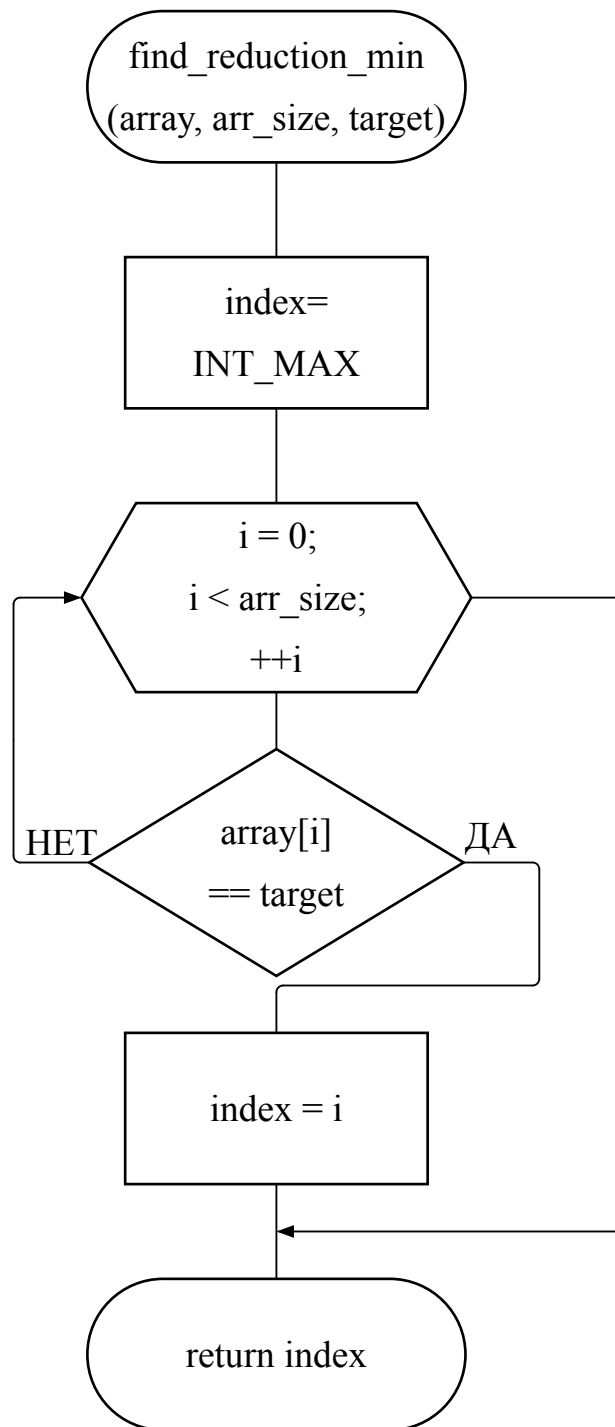


Рисунок 2 – Блок-схема параллельного алгоритма с отменой циклов (через `atomic` или `cancel for`)

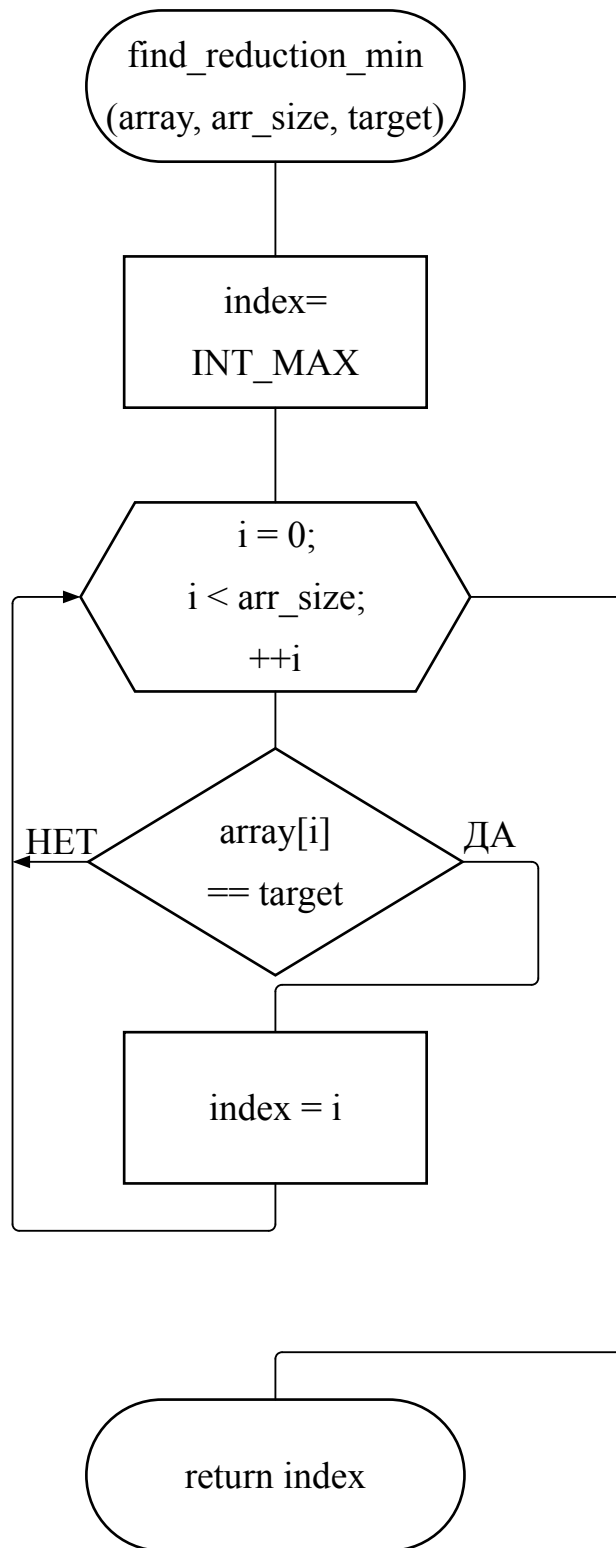


Рисунок 3 – Блок-схема параллельного алгоритма с применением редукции

5. Результаты экспериментов

Каждая точка на графике была получена путем повторения эксперимента 100 раз, на одинаковых исходных данных для каждого повтора. Случайные данные генерировались при помощи библиотеки c++ random.

Все алгоритмы были протестированы на двух массивах: array и array_close. Массив array из 10 млн. элементов, гарантированно никогда не содержал target. В array_close 1 млн элементов, 0.5% элементов были равны target, которые были случайно распределены по массиву.

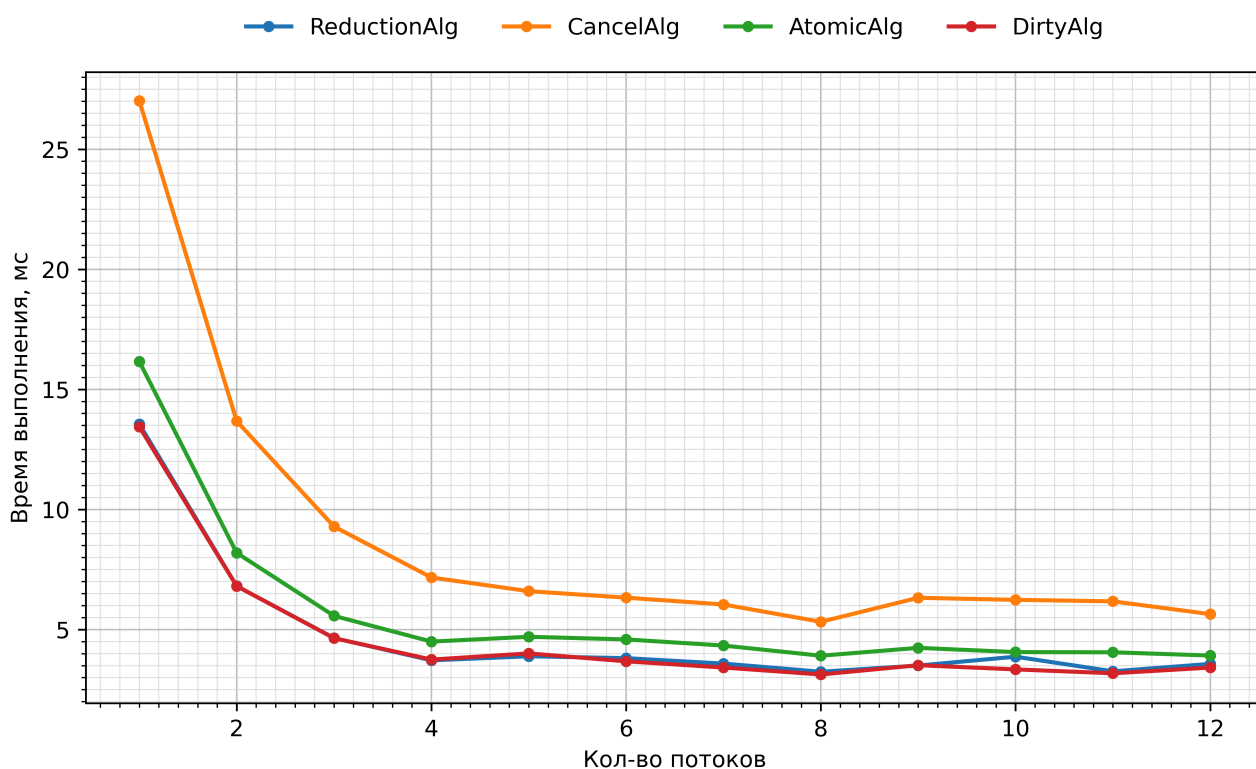


Рисунок 4 – График зависимости времени выполнения параллельных алгоритмов от количества потоков, на массиве array

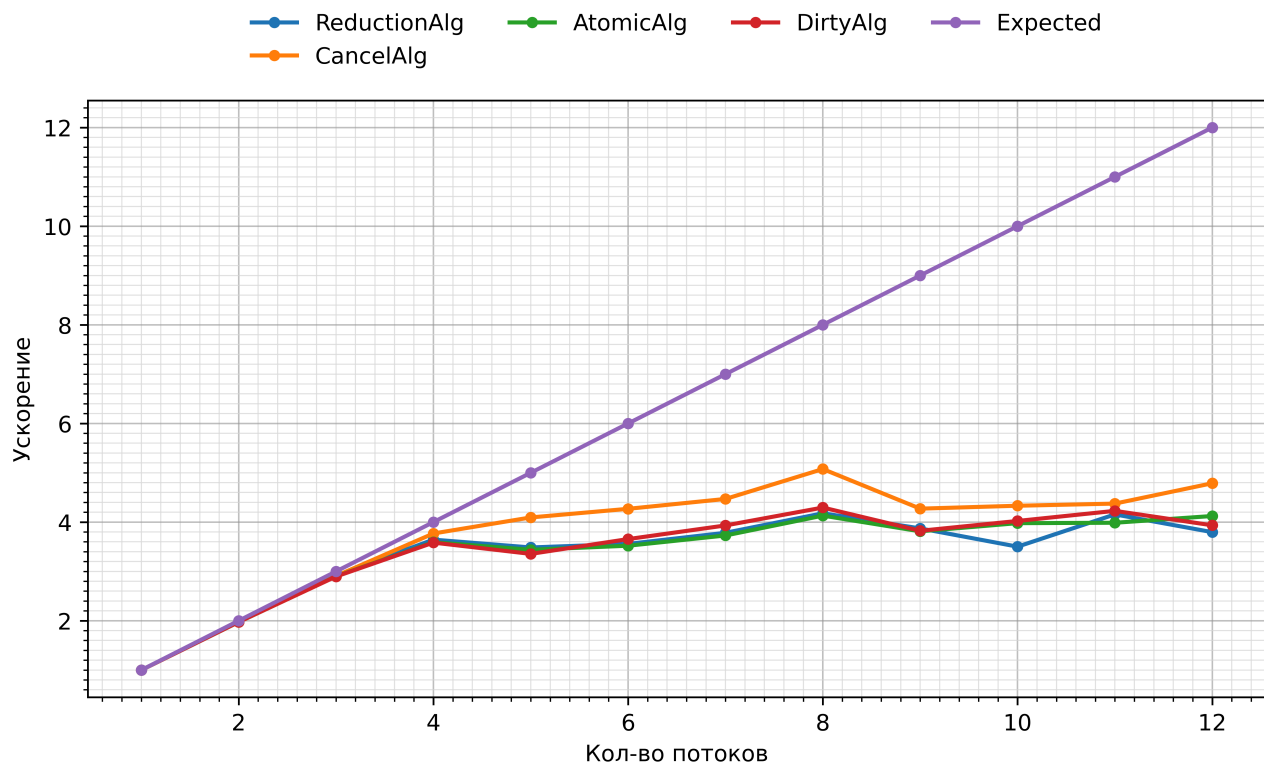


Рисунок 5 – График зависимости ускорения параллельных алгоритмов от количества потоков, на массиве array

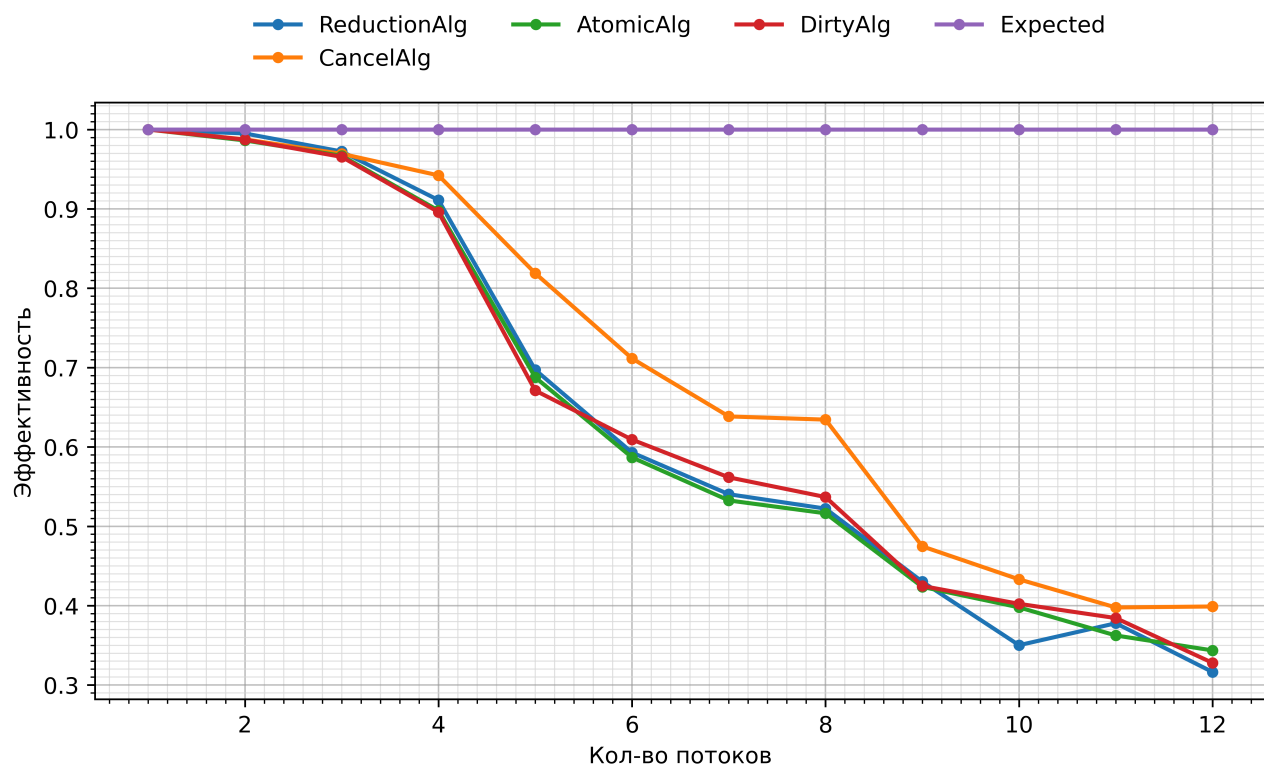


Рисунок 6 – График зависимости эффективности параллельных алгоритмов от количества потоков, на массиве array

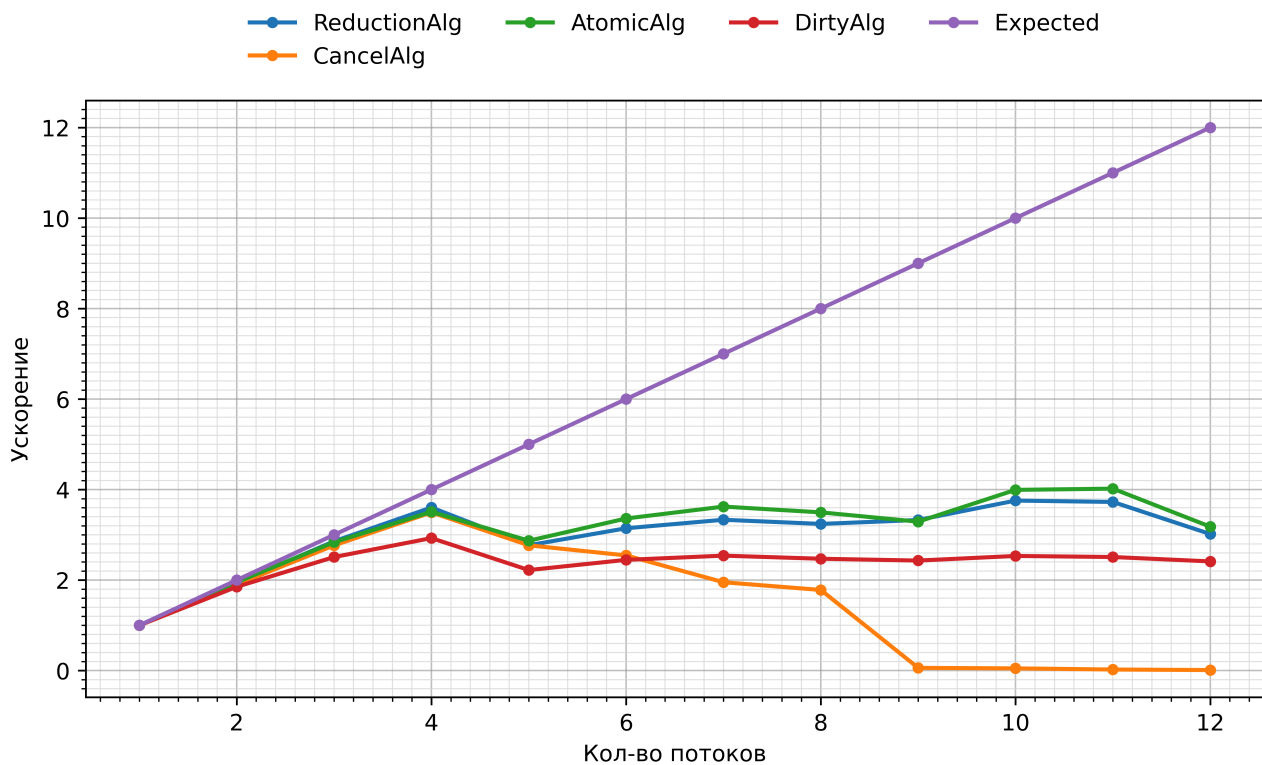


Рисунок 7 – График зависимости ускорения параллельных алгоритмов от количества потоков, на массиве array_close

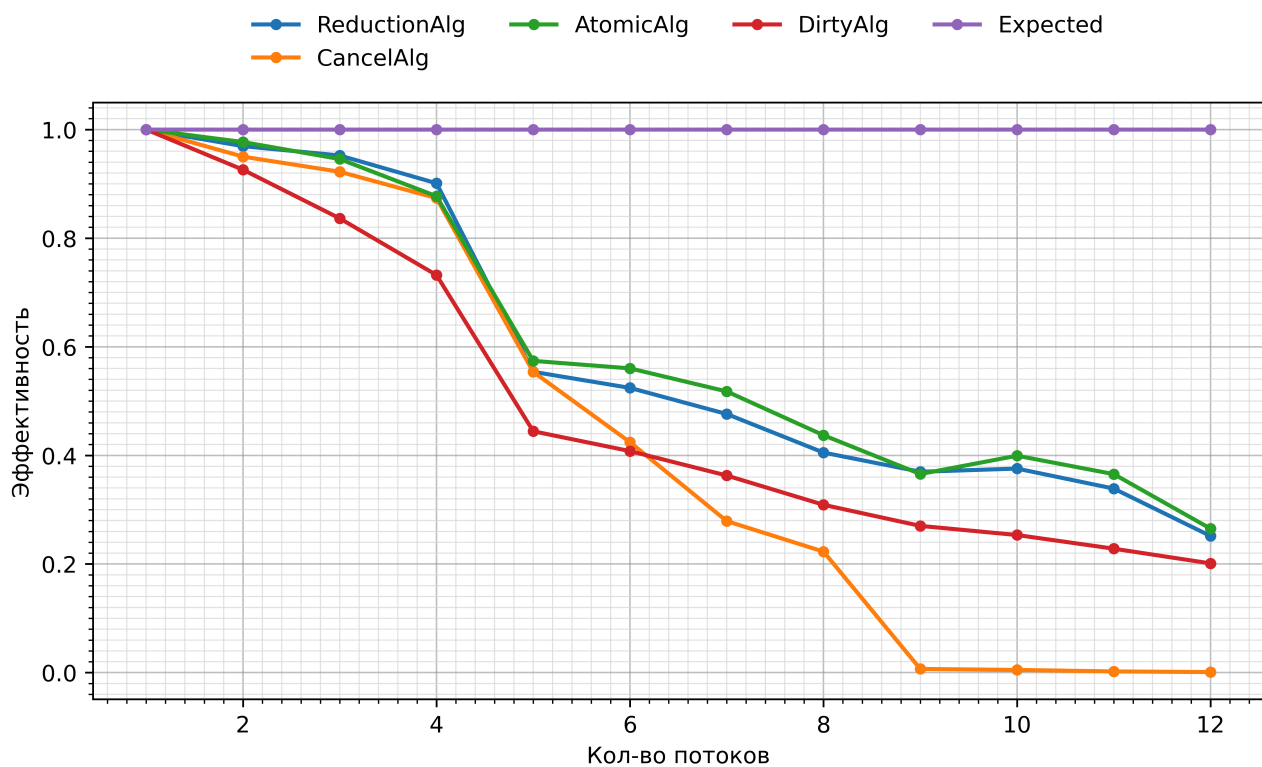


Рисунок 8 – График зависимости эффективности параллельных алгоритмов от количества потоков, на массиве array_close

6. Заключение

В ходе выполнения лабораторной работы были приобретены практические навыки выделения ресурса параллелизма в последовательном алгоритме и его реализации с использованием технологии OpenMP.

Был рассмотрен последовательный алгоритм поиска элемента в массиве, произведена оценка сложности: в лучшем случае – константная, в худшем – линейная. Построена блок-схема.

Выявлена область программы, где возможно применение параллельных вычислений, и реализованы 3 варианта алгоритма поиска:

1. Проход по всей длине массива с применением редукции (reduction)
2. Остановка при успешном сравнении (cancel)
3. Пропуск циклов при успешном сравнении и синхронизация через atomic (atomic)

Построены блок-схемы этих алгоритмов.

В качестве дополнительного задания был добавлен ещё один алгоритм (“Dirty”), в котором при успешном нахождении индекс в цикле for выставляется равным максимальному, что позволяет досрочно завершить циклы в других потоках. Этот метод был добавлен на все графики.

Экспериментально были получены значения времени работы, ускорения и эффективности для каждого из алгоритмов на двух различных массивах: array и array_close.

На массиве array, который гарантированно не содержал искомого элемента, все реализации, кроме cancel показали схожее время работы. На всех потоках cancel реализация оказалось примерно в два раза медленнее остальных. Это связано с накладными расходами на вызов проверки `#pragma omp cancellation point for` и более медленной синхронизацией через `critical`.

На массиве array_close, который гарантированно содержал 0.5% элементов, равных искомому, самыми быстрыми оказались atomic и reduction алгоритмы. dirty алгоритм показал аналогичные результаты на малом количестве потоков, но ускорение было меньше. Самым неэффективным оказался cancel. На 2-4 потоках алгоритм показывал в два раза большее время работы, на 4-8 потоках ускорение уменьшалось быстрее, чем для остальных алгоритмов. После 8

потоков эффективность распараллеливания стала стремиться к 0, а время работы выросло в десятки раз. Низкая эффективность cancel объясняется высокими затратами на синхронизацию и частыми обращениями к критическим секциям. При росте числа потоков эти накладные расходы быстро превышают выигрыш от параллелизма, что приводит к резкому росту времени выполнения и падению эффективности до нуля.

На графиках, как и в прошлой лабораторной, наблюдается снижение эффективности всех видов распараллеливания алгоритмов после достижения 4, 8 и 12 потоков. Это связано с устройством процессоров Apple M-серии (подробнее см. заключение лабораторной работы №1).

Таким образом, в работе достигнута цель построения параллельного алгоритма из последовательного с последующим анализом эффективности.

7. Приложение

7.1. Программные коды

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <omp.h>
#include <limits.h>
#include <random>

int32_t find_cancel(int32_t *arr, int32_t size, int32_t target, int32_t threads) {
    int32_t index = -1;

    #pragma omp parallel num_threads(threads) default(none) \
        shared(arr, size, target, index)
    {
        #pragma omp for
        for (int32_t i = 0; i < size; ++i) {
            if (arr[i] == target) {
                #pragma omp critical
                {
                    if (index == -1)
                        index = i;
                }
                #pragma omp cancel for
            }

            #pragma omp cancellation point for
        }
    }
    return index;
}
```

```

int32_t find_atomic(int32_t *arr, int32_t size, int32_t target, int32_t threads) {
    int32_t index = -1;

    #pragma omp parallel num_threads(threads) default(none) \
        shared(arr, size, target, index)
    {
        #pragma omp for
        for (int32_t i = 0; i < size; ++i) {
            int32_t current;
            #pragma omp atomic read
            current = index;
            if (current != -1) {
                continue;
            }

            if (arr[i] == target) {
                int32_t prev;
                #pragma omp atomic capture
                {
                    prev = index;
                    index = (prev == -1) ? i : prev;
                }
            }
        }
    }

    return index;
}

int32_t find_reduction_min(int32_t *arr, int32_t size, int32_t target, int32_t threads) {
    int32_t index = INT_MAX;

    #pragma omp parallel for num_threads(threads) \
        default(none) \
        shared(arr, size, target) \
        reduction(min:index)
    for (int32_t i = 0; i < size; ++i) {
        if (arr[i] == target) {
            if (i < index) index = i;
        }
    }

    return (index == INT_MAX) ? -1 : index;
}

int32_t find_dirty(int32_t *arr, int32_t size, int32_t target, int32_t threads) {
    int32_t index = -1;

    #pragma omp parallel for num_threads(threads) \
        default(none) \
        shared(arr, size, target, index)
    for (int32_t i = 0; i < size; ++i) {
        if (arr[i] == target) {
            index = i;
        }

        i = size;
    }
}

```

```

    }
}

return index;
}

void getExperimentDots(
    int32_t* array,
    int32_t sz,
    int32_t target,
    int32_t threads,
    int32_t repeats,
    FILE* file_res,
    FILE* file_speedup,
    FILE* file_eff,
    double* Tlcancel,
    double* Tlatomic,
    double* Tlreduction,
    double* Tldirty)
{
    double timeCancel = 0;
    double timeAtomic = 0;
    double timeReduction = 0;
    double timeDirty = 0;
    double start, end;
    for (int32_t i = 0; i < repeats; ++i) {
        start = omp_get_wtime();
        find_cancel(array, sz, target, threads);
        end = omp_get_wtime();
        timeCancel += end - start;
    }
    timeCancel /= repeats;
    timeCancel *= 1000;
    if (threads == 1) {
        *Tlcancel = timeCancel;
    }

    for (int32_t i = 0; i < repeats; ++i) {
        start = omp_get_wtime();
        find_atomic(array, sz, target, threads);
        end = omp_get_wtime();
        timeAtomic += end - start;
    }
    timeAtomic /= repeats;
    timeAtomic *= 1000;
    if (threads == 1) {
        *Tlatomic = timeAtomic;
    }

    for (int32_t i = 0; i < repeats; ++i) {
        start = omp_get_wtime();
        find_reduction_min(array, sz, target, threads);
        end = omp_get_wtime();
        timeReduction += end - start;
    }
    timeReduction /= repeats;
    timeReduction *= 1000;
}

```

```

    if (threads == 1) {
        *Tlreduction = timeReduction;
    }

    for (int32_t i = 0; i < repeats; ++i) {
        start = omp_get_wtime();
        find_dirty(array, sz, target, threads);
        end = omp_get_wtime();
        timeDirty += end - start;
    }
    timeDirty /= repeats;
    timeDirty *= 1000;
    if (threads == 1) {
        *Tldirty = timeDirty;
    }

    double realSpeedupCancel = (*Tlcancel) / timeCancel;
    double realSpeedupAtomic = (*Tlatomic) / timeAtomic;
    double realSpeedupReduction = (*Tlreduction) / timeReduction;
    double realSpeedupDirty = (*Tldirty) / timeDirty;
    if (file_res != NULL) {
        fprintf(file_res, "%d %lf %lf %lf %lf\n", threads, timeReduction, timeCancel, timeAtomic,
timeDirty);
    }
    if (file_speedup != NULL) {
        fprintf(file_speedup, "%d %lf %lf %lf %lf %d\n", threads, realSpeedupReduction,
realSpeedupCancel, realSpeedupAtomic, realSpeedupDirty, threads);
    }
    if (file_eff != NULL) {
        fprintf(file_eff, "%d %lf %lf %lf %lf %d\n", threads, realSpeedupReduction / threads,
realSpeedupCancel / threads, realSpeedupAtomic / threads, realSpeedupDirty/threads, 1);
    }
}

int32_t main(int32_t argc, char** argv)
{
    srand(time(NULL));

    FILE* file_res = fopen("results/runtime_array", "w");
    fprintf(file_res, "4\n");
    fprintf(file_res, "ReductionAlg\n");
    fprintf(file_res, "CancelAlg\n");
    fprintf(file_res, "AtomicAlg\n");
    fprintf(file_res, "DirtyAlg\n");
    fprintf(file_res, "Кол-во потоков\n");
    fprintf(file_res, "Время выполнения, мс\n");
    fprintf(file_res, "none\n");

    FILE* file_speedup = fopen("results/speedup_array", "w");
    fprintf(file_speedup, "5\n");
    fprintf(file_speedup, "ReductionAlg\n");
    fprintf(file_speedup, "CancelAlg\n");
    fprintf(file_speedup, "AtomicAlg\n");
    fprintf(file_speedup, "DirtyAlg\n");
    fprintf(file_speedup, "Expected\n");
    fprintf(file_speedup, "Кол-во потоков\n");
    fprintf(file_speedup, "Ускорение\n");

```

```

fprintf(file_speedup, "none\n");

FILE* file_eff = fopen("results/efficiency_array", "w");
fprintf(file_eff, "5\n");
fprintf(file_eff, "ReductionAlg\n");
fprintf(file_eff, "CancelAlg\n");
fprintf(file_eff, "AtomicAlg\n");
fprintf(file_eff, "DirtyAlg\n");
fprintf(file_eff, "Expected\n");
fprintf(file_eff, "Кол-во потоков\n");
fprintf(file_eff, "Эффективность\n");
fprintf(file_eff, "none\n");

FILE* file_res_close = fopen("results/runtime_close", "w");
fprintf(file_res_close, "4\n");
fprintf(file_res_close, "ReductionAlg\n");
fprintf(file_res_close, "CancelAlg\n");
fprintf(file_res_close, "AtomicAlg\n");
fprintf(file_res_close, "DirtyAlg\n");
fprintf(file_res_close, "Кол-во потоков\n");
fprintf(file_res_close, "Время выполнения, мс\n");
fprintf(file_res_close, "none\n");

FILE* file_speedup_close = fopen("results/speedup_close", "w");
fprintf(file_speedup_close, "5\n");
fprintf(file_speedup_close, "ReductionAlg\n");
fprintf(file_speedup_close, "CancelAlg\n");
fprintf(file_speedup_close, "AtomicAlg\n");
fprintf(file_speedup_close, "DirtyAlg\n");
fprintf(file_speedup_close, "Expected\n");
fprintf(file_speedup_close, "Кол-во потоков\n");
fprintf(file_speedup_close, "Ускорение\n");
fprintf(file_speedup_close, "none\n");

FILE* file_eff_close = fopen("results/efficiency_close", "w");
fprintf(file_eff_close, "5\n");
fprintf(file_eff_close, "ReductionAlg\n");
fprintf(file_eff_close, "CancelAlg\n");
fprintf(file_eff_close, "AtomicAlg\n");
fprintf(file_eff_close, "DirtyAlg\n");
fprintf(file_eff_close, "Expected\n");
fprintf(file_eff_close, "Кол-во потоков\n");
fprintf(file_eff_close, "Эффективность\n");
fprintf(file_eff_close, "none\n");

int32_t sz = pow(10, 7);
int32_t sz_close = pow(10, 6);
int32_t target_frequency = 0.005 * sz_close;
int32_t repeats = 100;
int32_t max_threads = 12;
double Tlreduction, Tlatomic, Tlcancel, Tldirty;
double Tlreduction_close, Tlatomic_close, Tlcancel_close, Tldirty_close;

int32_t* array = (int32_t*)malloc(sz*sizeof(int32_t));
int32_t* array_close = (int32_t*)malloc(sz_close*sizeof(int32_t));

std::mt19937_64 rng(123);

```

```

std::uniform_int_distribution<size_t> idx(0, sz_close-1);
std::uniform_int_distribution<int32_t> target_random(0, 10000000);
for (int32_t threads=1; threads <= max_threads; ++threads) {
    int32_t target = target_random(rng);
    int32_t not_target = target - 1;
    for(int32_t i=0; i<sz; i++) {
        array[i] = not_target;
    }
    for(int32_t i=0; i<sz_close; i++) {
        array_close[i] = not_target;
    }

    for (int i = 0; i < target_frequency; ++i) {
        array_close[idx(rng)] = target;
    }

    getExperimentDots(array, sz, target, threads, repeats, file_res, file_speedup, file_eff,
&Tlcancel, &Tlatomic, &Tlreduction, &Tldirty);
    getExperimentDots(array_close, sz_close, target, threads, repeats, file_res_close,
file_speedup_close, file_eff_close, &Tlcancel_close, &Tlatomic_close, &Tlreduction_close,
&Tldirty_close);

    printf("finished %d\n", threads);
}

free(array);
free(array_close);
fclose(file_res);
fclose(file_speedup);
fclose(file_eff);
return 0;
}

```

Файл 1 – Замер времени, вычисление ускорения и эффективности для параллельных алгоритмов

7.2. Таблицы

Таблица 1 – Сравнение теоретических и экспериментальных данных

	Теор. значения	Редукция	pragma cancel for	atomic	dirty
Ускорение	P	1.99 – 4.18	1.98 – 5.08	1.97 – 4.13	1.97 – 4.29
Эффективность	P	0.32 – 0.99	0.40 – 0.99	0.34 – 0.99	0.33 – 0.00

Таблица 2 – Данные о времени работы параллельных алгоритмов, на массиве array

Кол-во потоков	ReductionAlg	CancelAlg	AtomicAlg	DirtyAlg
1	13.554330	27.018168	16.160090	13.444495
2	6.810920	13.678980	8.192601	6.808751
3	4.646678	9.289858	5.571170	4.641442
4	3.719239	7.168701	4.499271	3.751311
5	3.888421	6.598949	4.699817	4.006820
6	3.809710	6.328521	4.589810	3.677952
7	3.582871	6.045177	4.334459	3.419249
8	3.243930	5.323260	3.911812	3.130331
9	3.500710	6.323612	4.238341	3.516140
10	3.870542	6.237729	4.062028	3.341911
11	3.261359	6.175580	4.053540	3.179848
12	3.570623	5.643482	3.919232	3.416841

Таблица 3 – Данные об ускорении параллельных алгоритмов, на массиве array

Кол-во потоков	ReductionAlg	CancelAlg	AtomicAlg	DirtyAlg	Expected
1	1.000000	1.000000	1.000000	1.000000	1
2	1.990088	1.975160	1.972522	1.974591	2
3	2.916994	2.908351	2.900664	2.896620	3
4	3.644382	3.768907	3.591713	3.583945	4
5	3.485818	4.094314	3.438451	3.355403	5
6	3.557838	4.269271	3.520862	3.655429	6
7	3.783092	4.469376	3.728283	3.932002	7
8	4.178367	5.075493	4.131101	4.294912	8
9	3.871881	4.272585	3.812834	3.823652	9
10	3.501920	4.331411	3.978331	4.022996	10
11	4.156038	4.375001	3.986661	4.228031	11
12	3.796068	4.787499	4.123280	3.934773	12

Таблица 4 – Данные об эффективности параллельных алгоритмов, на массиве array

Кол-во потоков	ReductionAlg	CancelAlg	AtomicAlg	DirtyAlg	Expected
1	1.000000	1.000000	1.000000	1.000000	1
2	0.995044	0.987580	0.986261	0.987295	1
3	0.972331	0.969450	0.966888	0.965540	1
4	0.911096	0.942227	0.897928	0.895986	1
5	0.697164	0.818863	0.687690	0.671081	1
6	0.592973	0.711545	0.586810	0.609238	1
7	0.540442	0.638482	0.532612	0.561715	1
8	0.522296	0.634437	0.516388	0.536864	1
9	0.430209	0.474732	0.423648	0.424850	1
10	0.350192	0.433141	0.397833	0.402300	1
11	0.377822	0.397727	0.362424	0.384366	1
12	0.316339	0.398958	0.343607	0.327898	1

Таблица 5 – Данные о времени работы параллельных алгоритмов, на массиве array_close

Кол-во потоков	ReductionAlg	CancelAlg	AtomicAlg	DirtyAlg
1	1.375759	2.777371	1.353111	1.366811
2	0.709579	1.461649	0.692506	0.738008
3	0.481591	1.003819	0.476952	0.544722
4	0.381691	0.794649	0.385649	0.466769
5	0.496769	1.003110	0.471439	0.615101
6	0.437372	1.091101	0.402548	0.558517
7	0.412750	1.422389	0.373349	0.538049
8	0.424454	1.558492	0.386941	0.552940
9	0.413351	45.802581	0.411150	0.562122
10	0.366180	56.349487	0.338800	0.539491
11	0.369139	117.467060	0.336571	0.544543
12	0.455868	223.739207	0.425432	0.566421

Таблица 6 – Данные об ускорении параллельных алгоритмов, на массиве array_close

Кол-во потоков	ReductionAlg	CancelAlg	AtomicAlg	DirtyAlg	Expected
1	1.000000	1.000000	1.000000	1.000000	1
2	1.938838	1.900163	1.953935	1.852027	2
3	2.856694	2.766806	2.836999	2.509189	3
4	3.604374	3.495092	3.508658	2.928240	4
5	2.769414	2.768759	2.870170	2.222092	5
6	3.145513	2.545476	3.361364	2.447214	6
7	3.333150	1.952611	3.624251	2.540310	7
8	3.241242	1.782089	3.496941	2.471895	8
9	3.328306	0.060638	3.291037	2.431520	9
10	3.757056	0.049288	3.993835	2.533518	10
11	3.726942	0.023644	4.020288	2.510013	11
12	3.017887	0.012413	3.180560	2.413067	12

Таблица 7 – Данные об эффективности параллельных алгоритмов, на массиве array_close

Кол-во потоков	ReductionAlg	CancelAlg	AtomicAlg	DirtyAlg	Expected
1	1.000000	1.000000	1.000000	1.000000	1
2	0.969419	0.950082	0.976967	0.926014	1
3	0.952231	0.922269	0.945666	0.836396	1
4	0.901093	0.873773	0.877165	0.732060	1
5	0.553883	0.553752	0.574034	0.444418	1
6	0.524252	0.424246	0.560227	0.407869	1
7	0.476164	0.278944	0.517750	0.362901	1
8	0.405155	0.222761	0.437118	0.308987	1
9	0.369812	0.006738	0.365671	0.270169	1
10	0.375706	0.004929	0.399384	0.253352	1
11	0.338813	0.002149	0.365481	0.228183	1
12	0.251491	0.001034	0.265047	0.201089	1